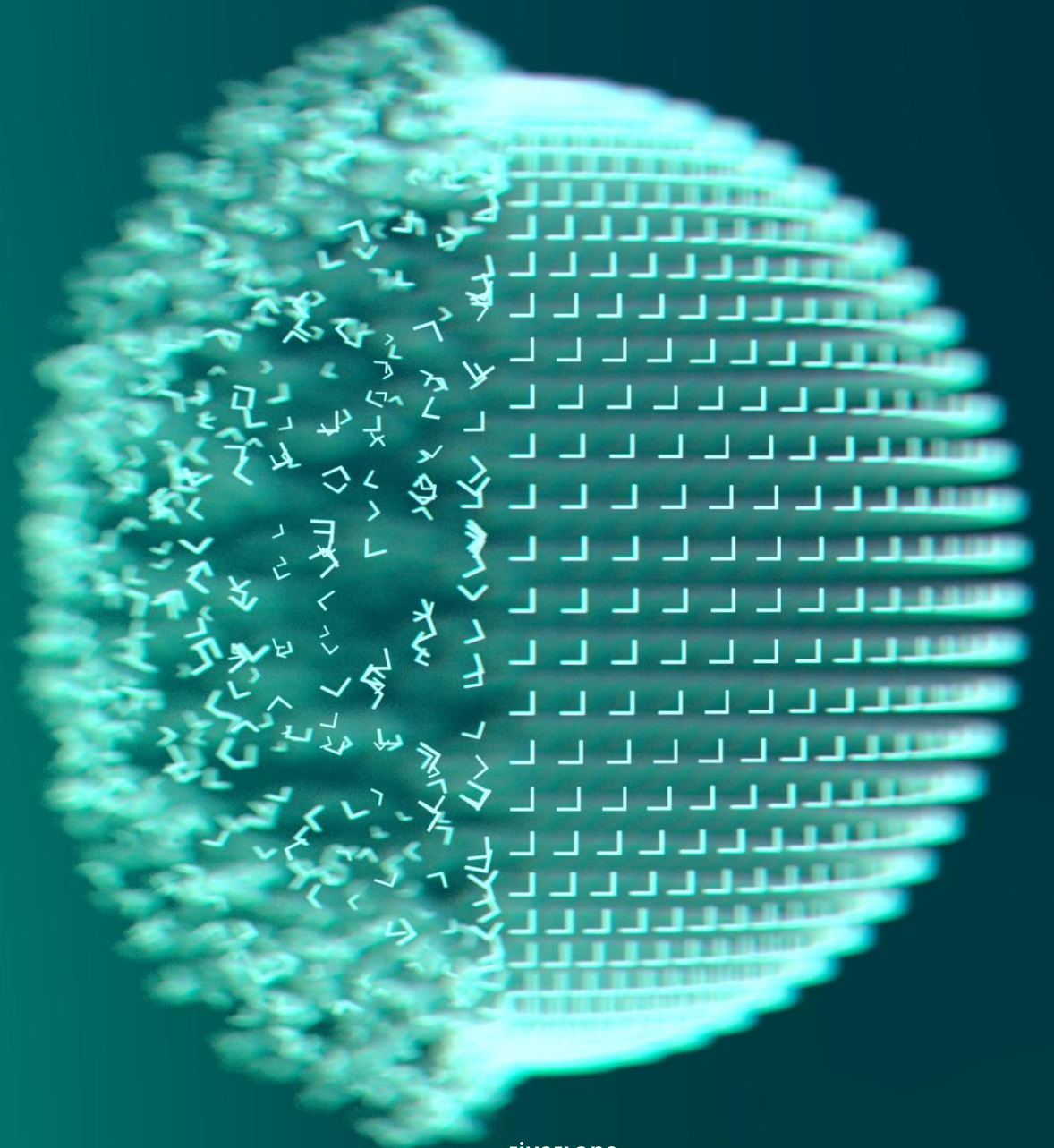


# Compilers at Riverlane

Adam Melvin, Staff Software Architect



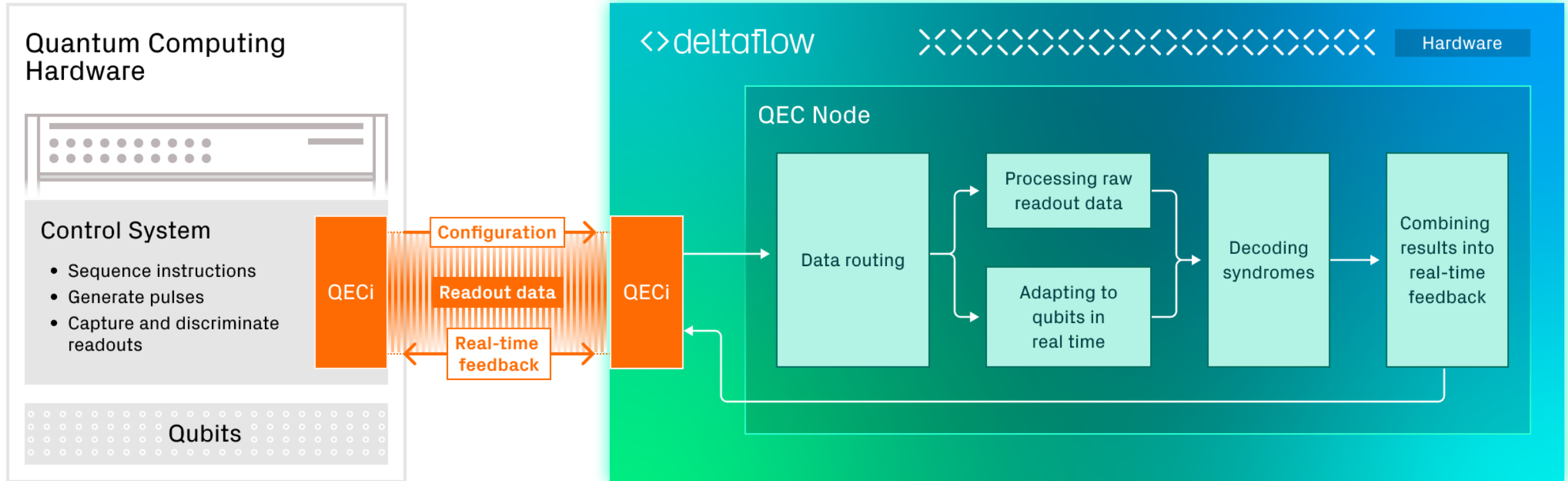
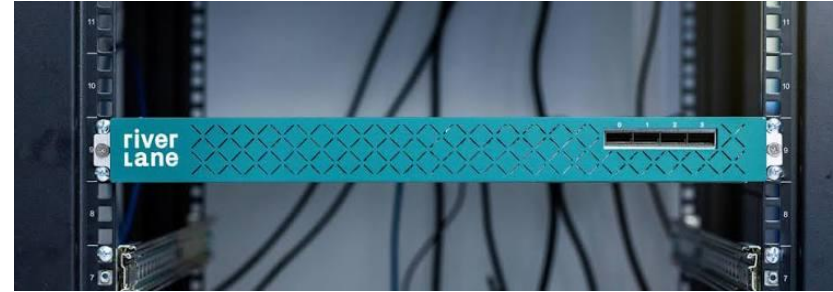
# Plan of the talk

- 00 Setting the Problem
- 01 Riverlane Software Overview
- 02 Dynamic DEMs

# Setting the Problem

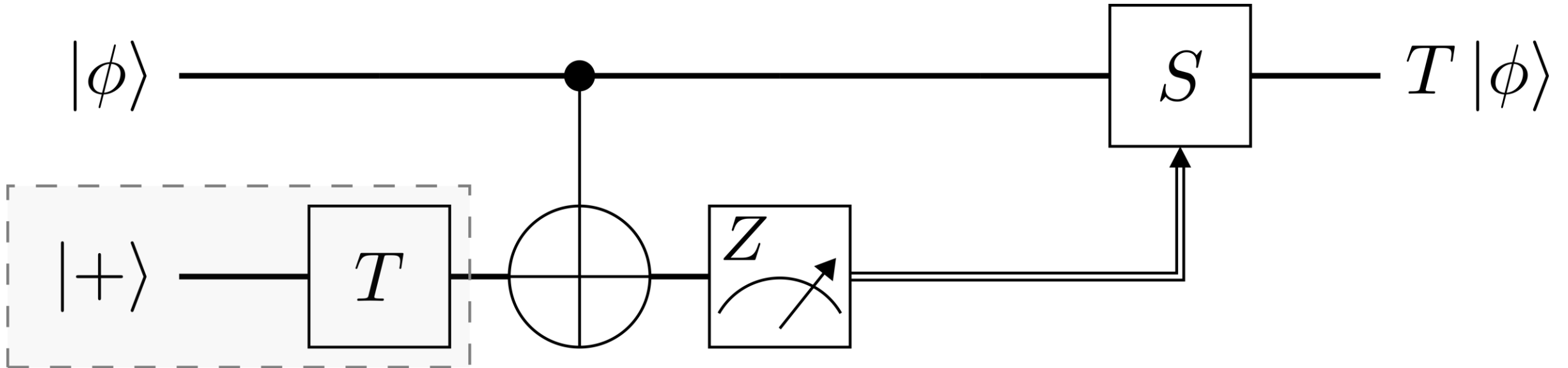
# Introduction

## Hardware QEC system

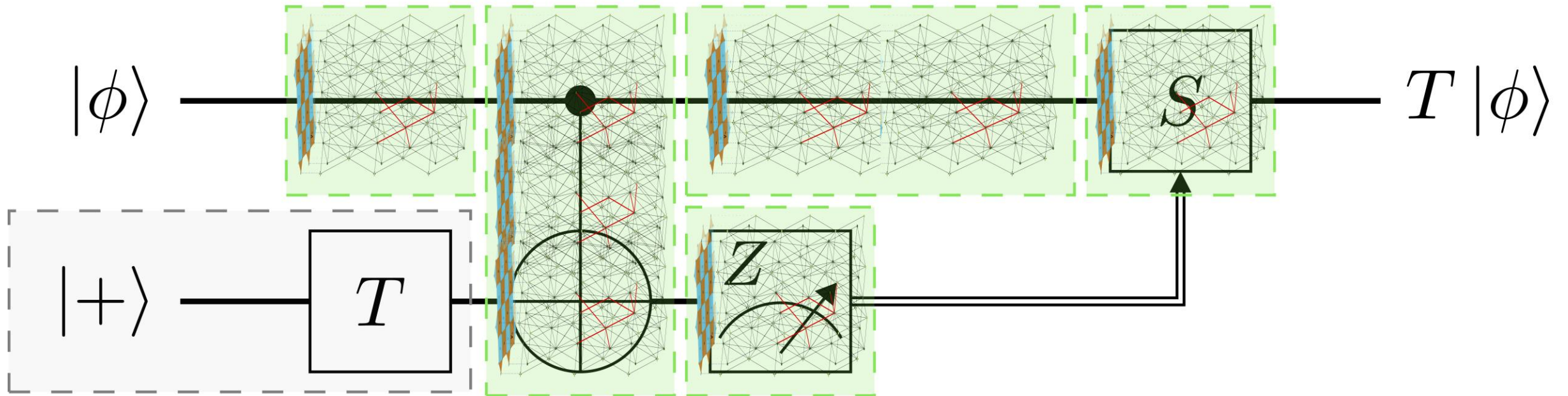


# Magic state teleportation

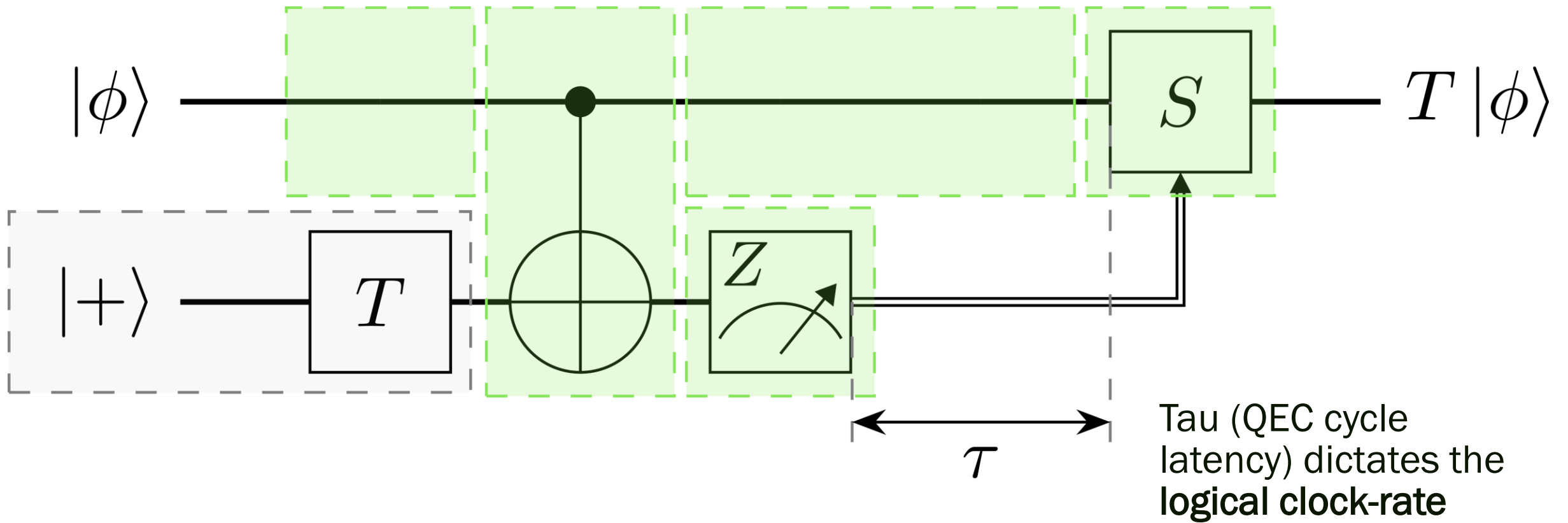
## Non-Clifford T gate



## Windowed decoding

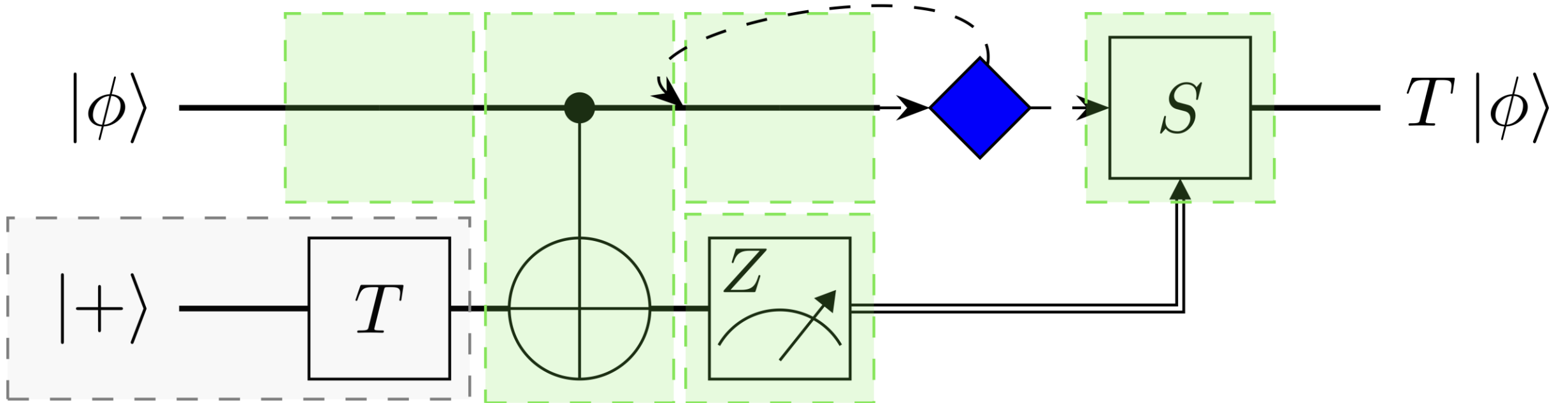


## Real time decoding and latency



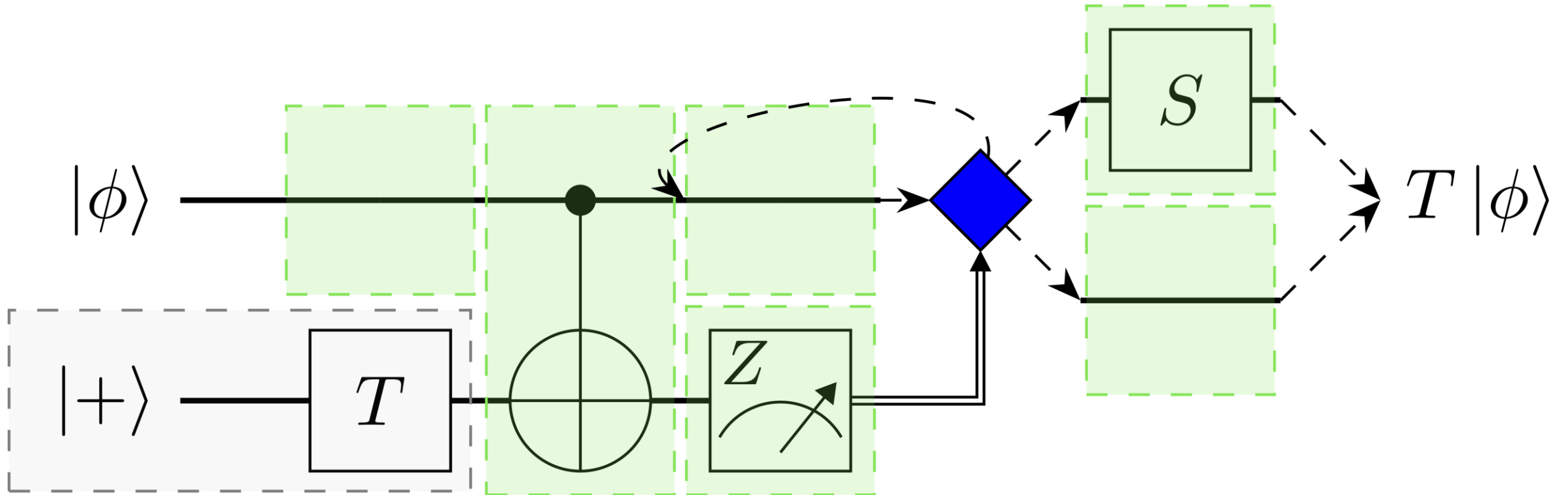
## Branching due to idling

Explicit control flow lets continue to measure stabilisers until decoding is finished



## Branching

Explicit control flow lets us decode differently based on runtime information

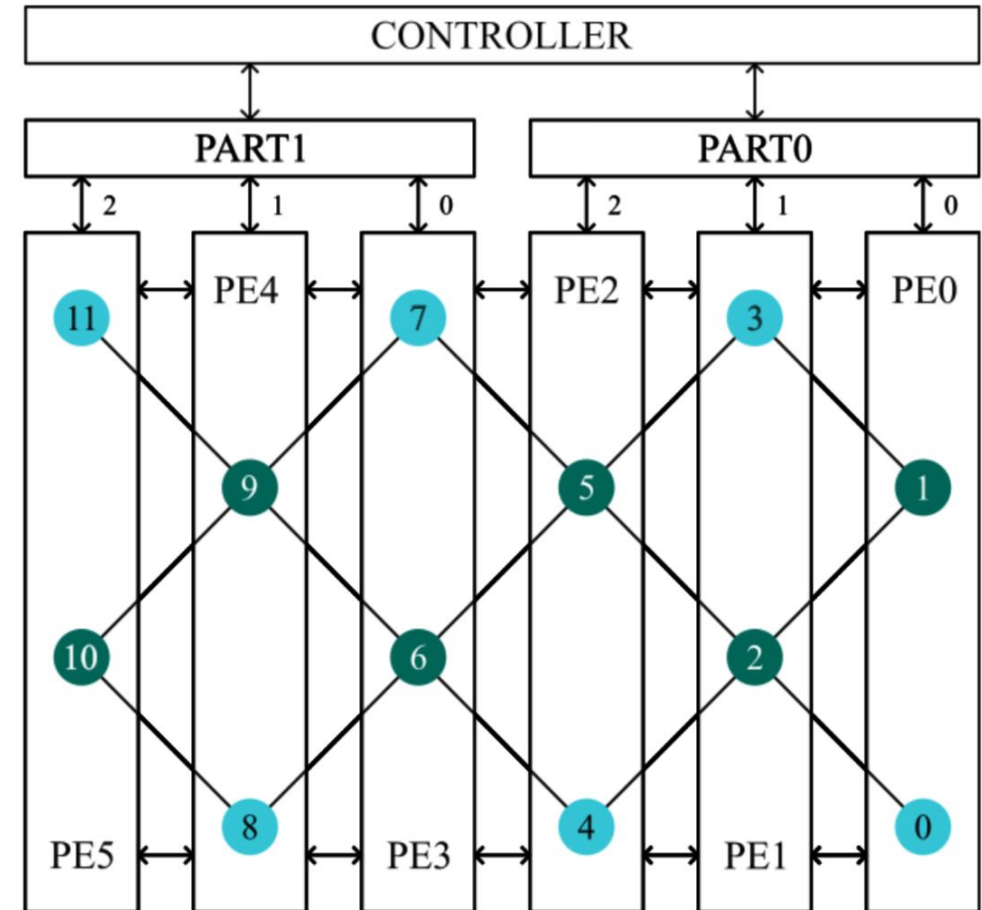
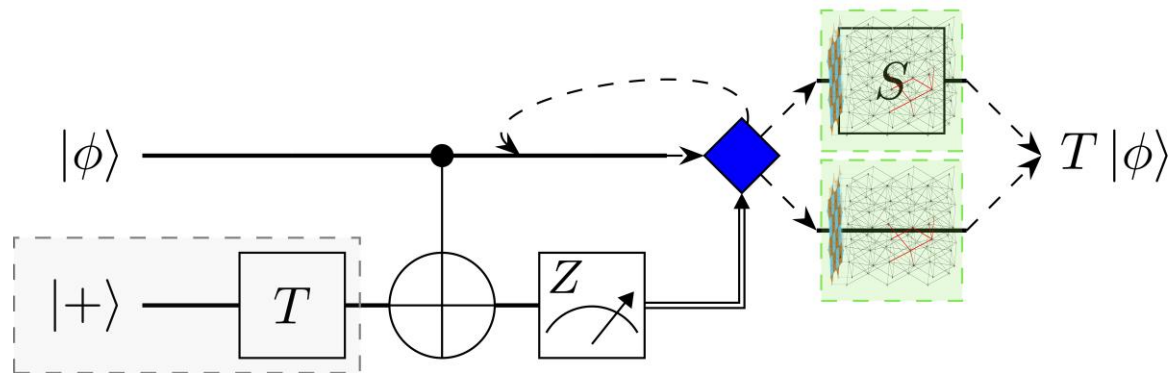


- Multiple execution paths

# What is hard about decoding different windows

Explicit control flow lets us choose which window to decode

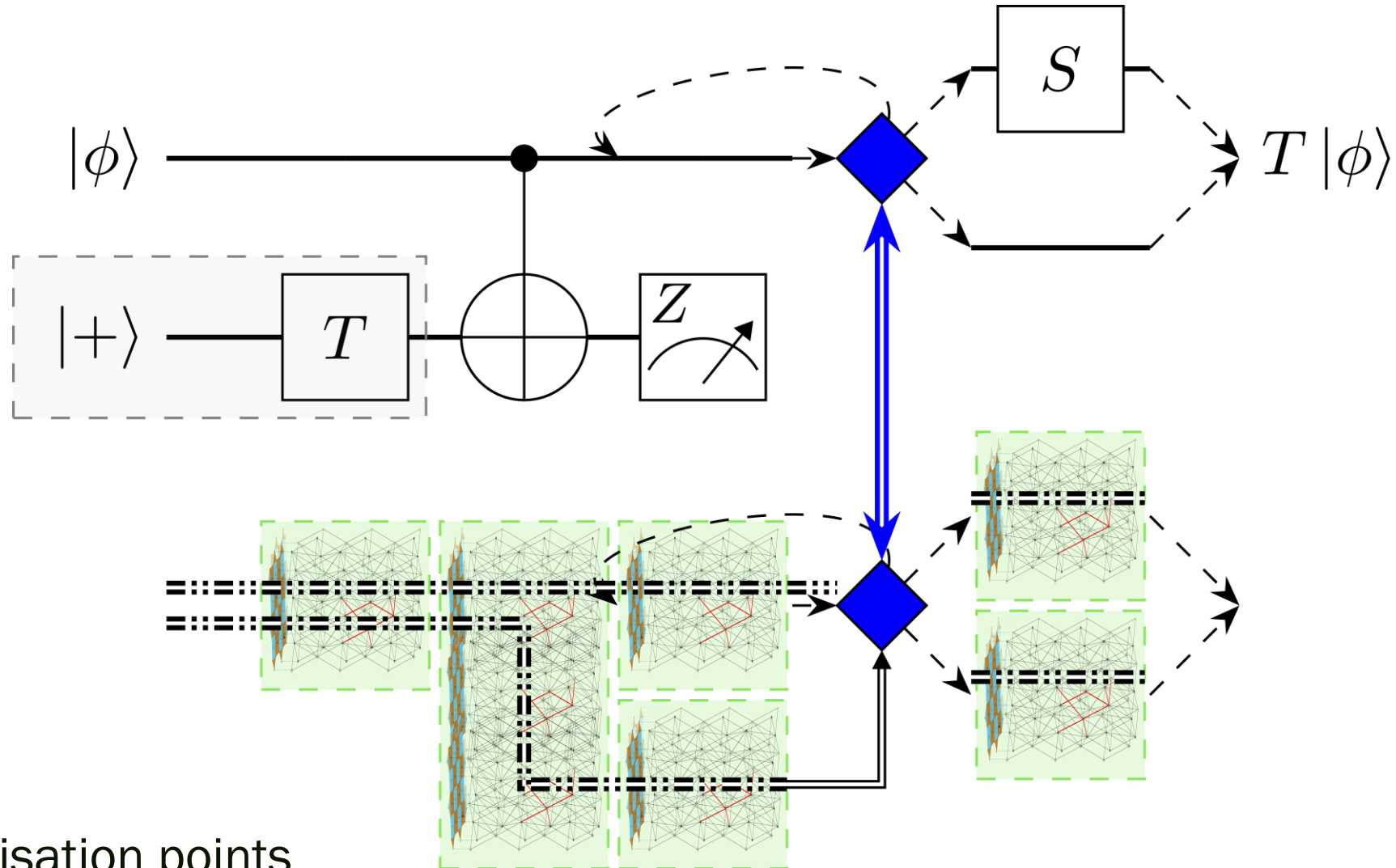
- QEC system needs to be configured with awareness of decoding graph
- Branching introduces multiple decoding graphs that the QEC system needs to be ready to switch between depending on real-time results
- Loading decoding graphs takes time!



PE Diagram from Ziad, A.B., Zalawadiya, A., Topal, C. et al. Local clustering decoder as a fast and adaptive hardware decoder for the surface code. Nat Commun 16, 11048 (2025). <https://doi.org/10.1038/s41467-025-66773-x>

# Synchronicity

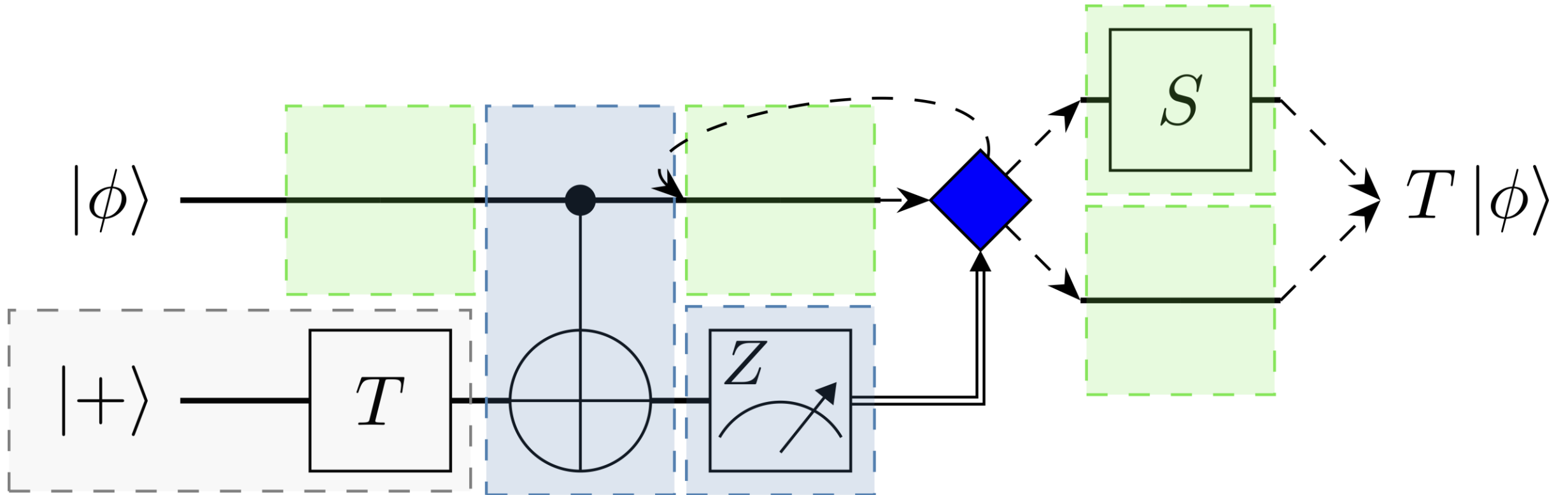
Control system and decoder execution are derived from the same program



- Synchronisation points

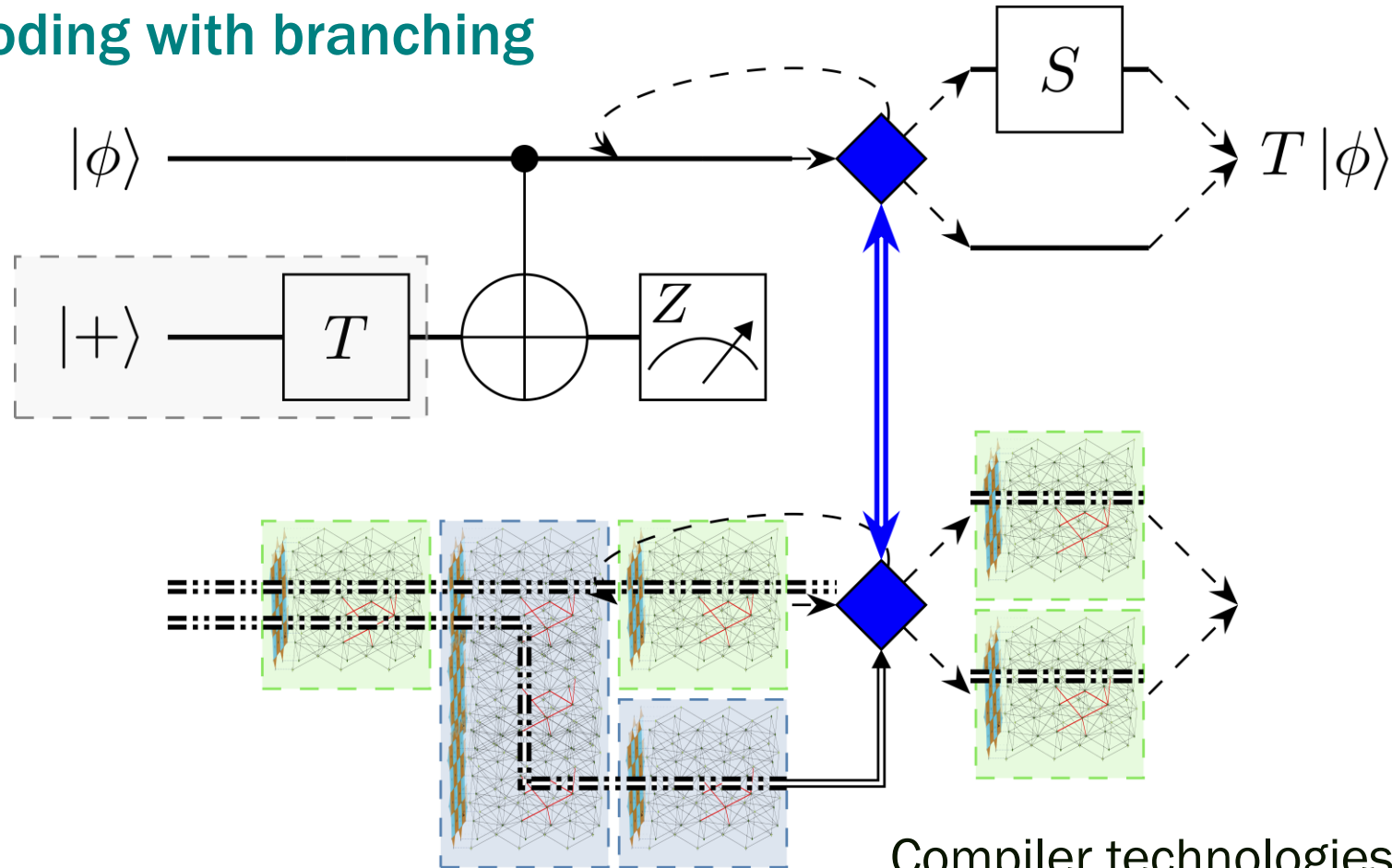
## Parallelism

Explicit control flow gives us opportunities to parallelise decoding



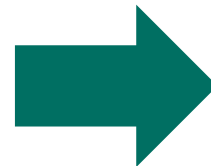
- Decoder parallelism opportunities

## Windowed decoding with branching



We now have

- Multiple execution paths
- Synchronization points (QEC and ctrlsys)
- Opportunities for parallel execution.



Compiler technologies have frameworks

- Control flow
- Scheduling
- Parallelism

# Riverlane Software Overview

## Deltaflow Tools

### *Tools for system integrators*

The “driver layer” for Deltaflow – facilitating:

- System integration
- Test and debug
- System configuration
- Low-level compilation

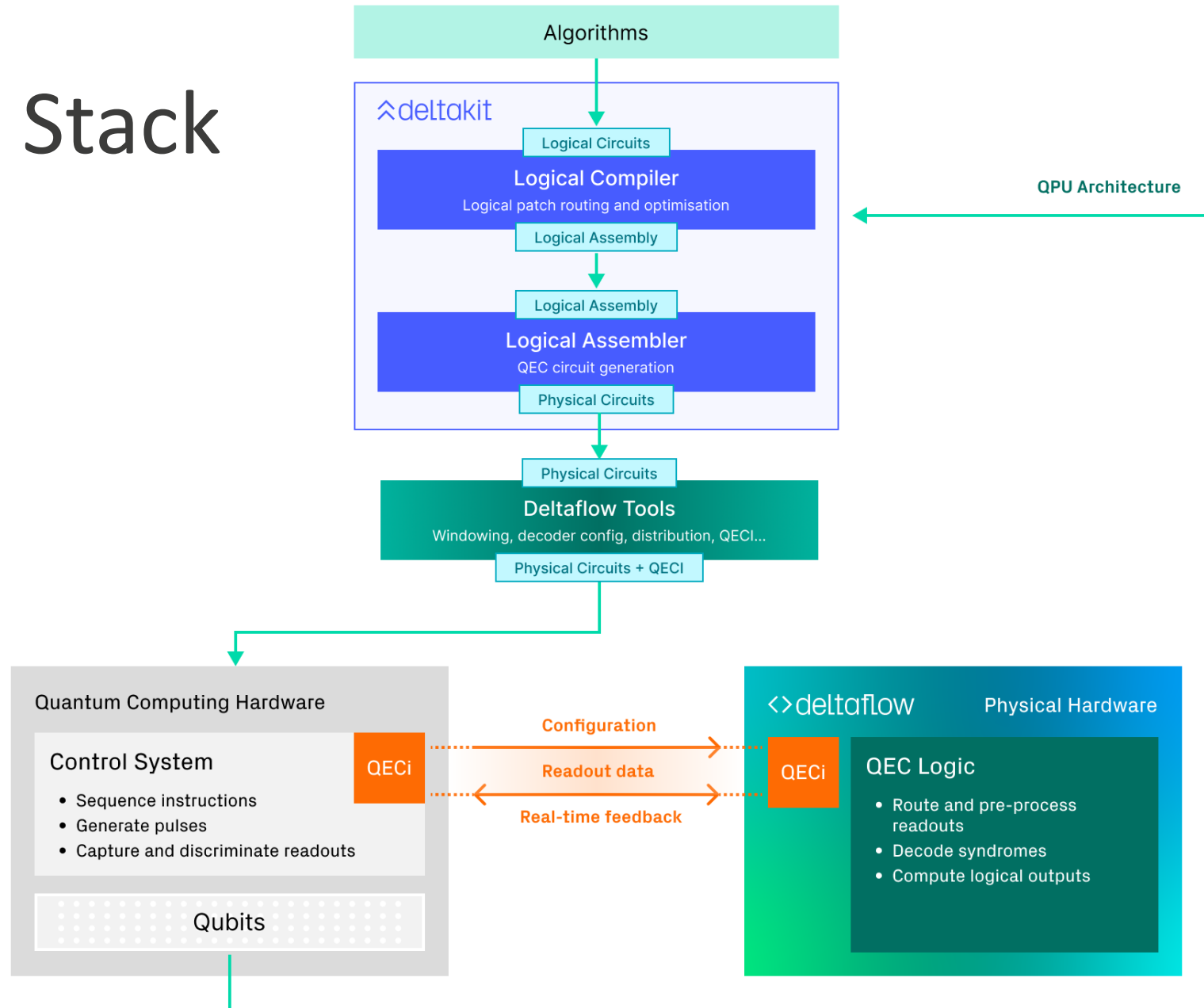
## Deltakit

### *Tools for system end users*

The “application layer” for Deltaflow, allowing:

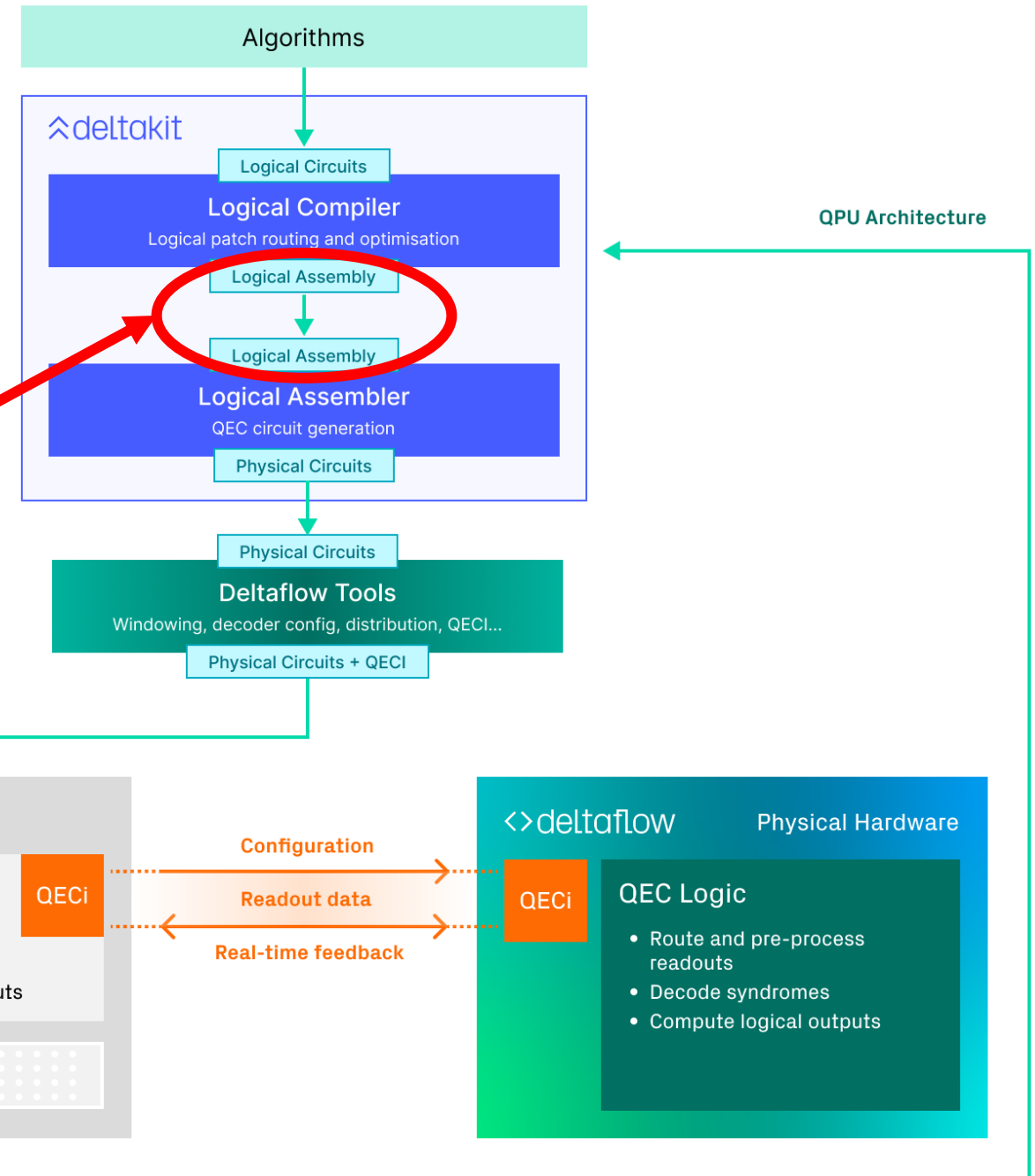
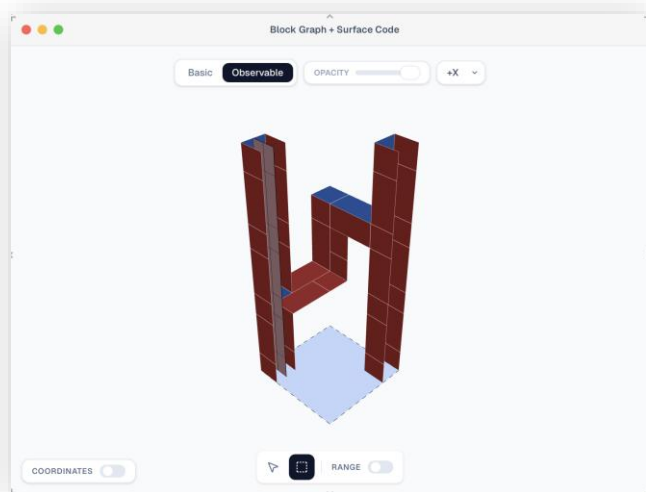
- QEC Code exploration
- High level compilation
- Circuit visualisation
- QEC results analysis

# The QEC Stack



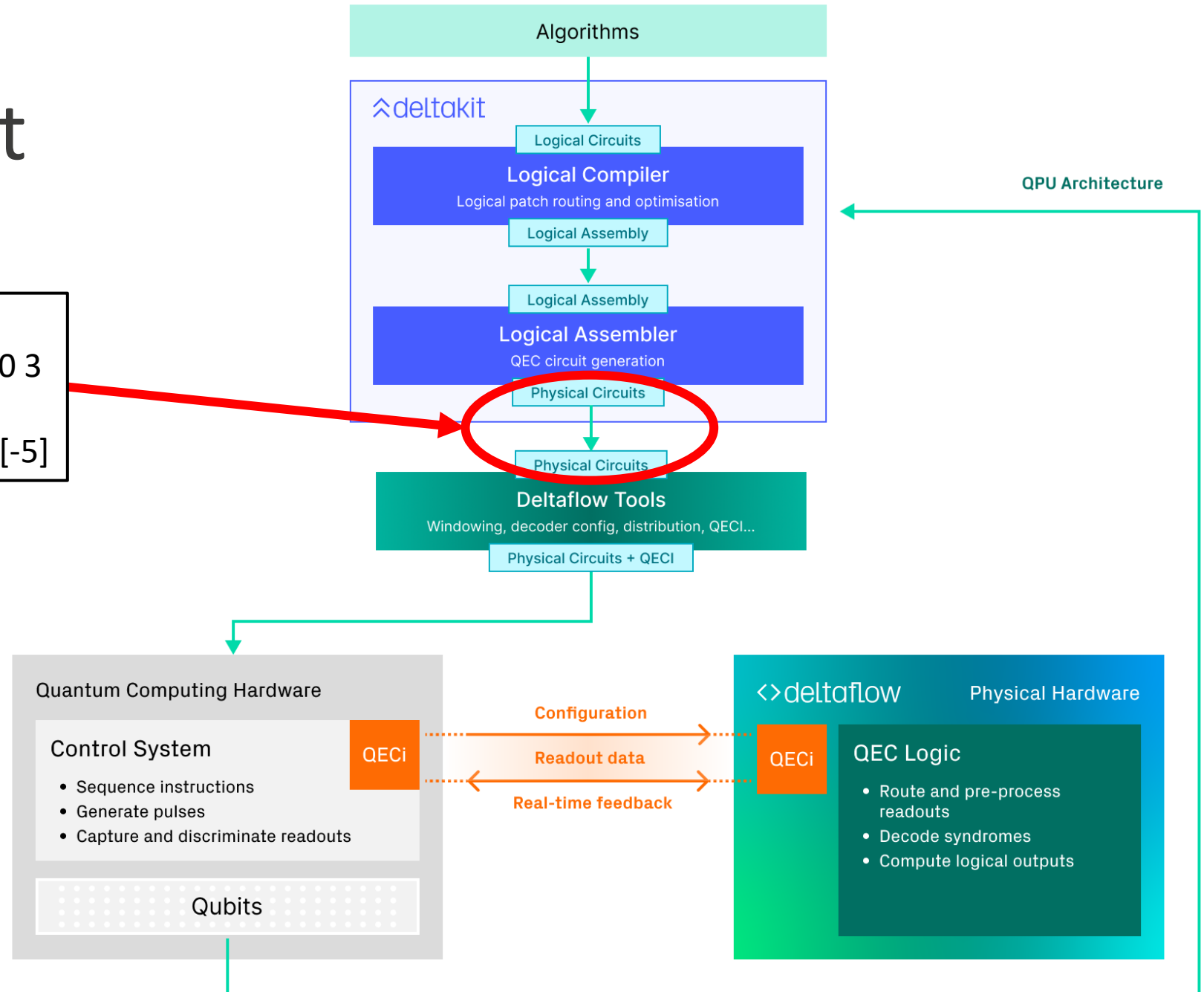
# Logical Assembly

```
1 d = 5
2 builder = LogAsmBuilder()
3 p0 = builder.declare_patch(RotatedPlanarPatch(d, d))
4 p1 = builder.declare_patch(RotatedPlanarPatch(d, d))
5
6 p0.prepare(Z)
7 p1.prepare(Z)
8 p0.measure_stabilisers(8 * d)
9 p1.measure_stabilisers(8 * d)
10
11 b0 = p0.measure(Z)
12 with builder.if_(b0 == 1):
13     p1.transversal(X)
14
15 b1 = p1.measure(Z)
16
17 Compiler().compile(builder.build())
```



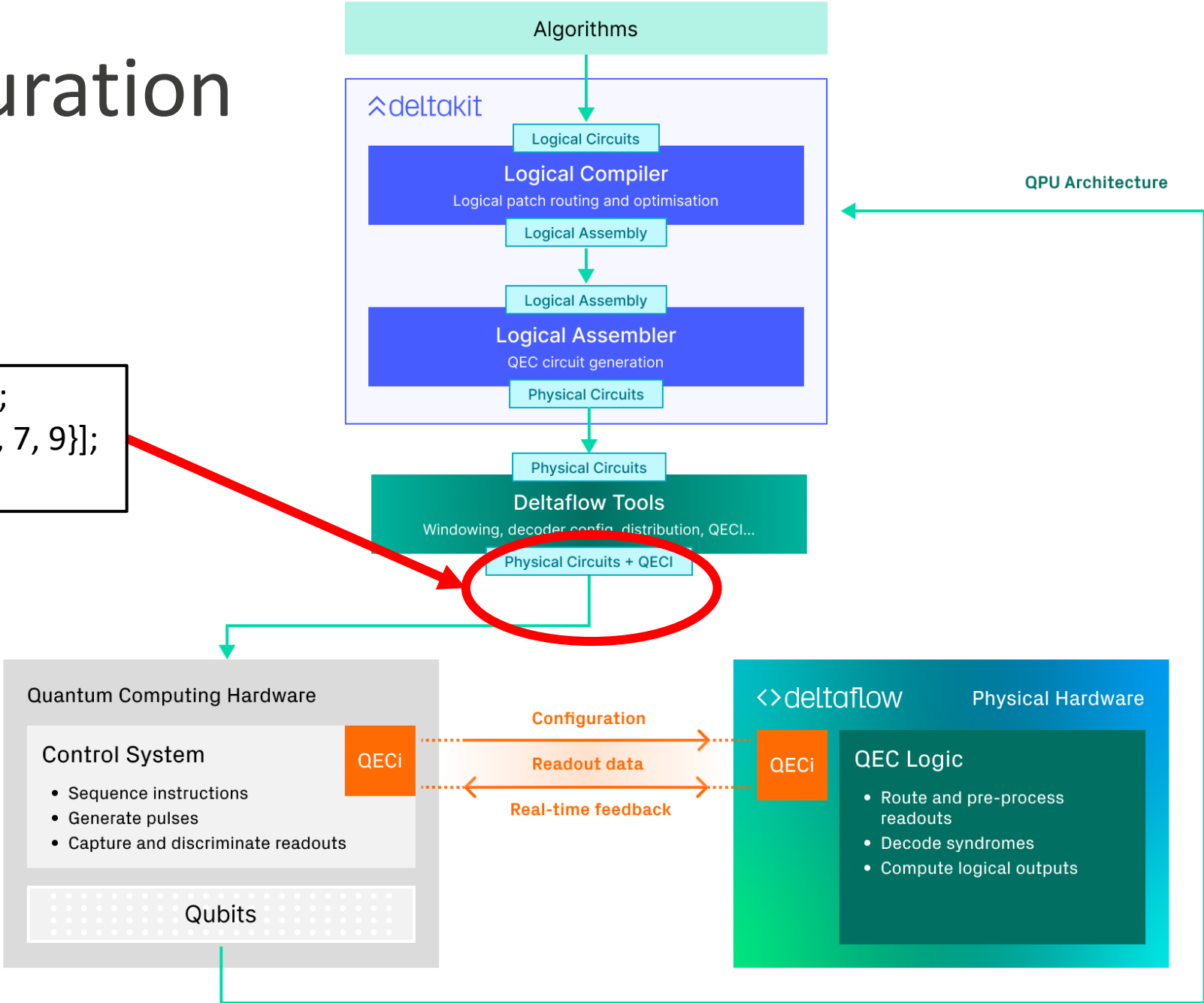
# Physical Circuit

```
CZ 0 1 3 5 7 6 9 10
DEPOLARIZE1(0.001) 0 3
M 0 3 7 9
DETECTOR rec[-1] rec[-5]
```



# Adding configuration for Deltaflow

```
cz q[{0, 1, 3, 5, 7, 6, 9, 10}];  
bit[4] ro = measure q[{0, 3, 7, 9}];  
qeci_readout_send(..., ro);
```



# Current circuit representations

Stim

```
CZ 0 1 3 5 7 6 9 10  
DEPOLARIZE1(0.001) 0 3  
M 0 3 7 9  
DETECTOR rec[-1] rec[-5]
```

Lacking classical operations

OpenQASM 3

```
cz q[{0, 1, 3, 5, 7, 6, 9, 10}];  
bit[4] ro = measure q[{0, 3, 7, 9}];  
if (ro[0] == 1) {  
    x q[0];  
}
```

Lacking QEC operations

# MLIR and Intermediate Representations (IR)

```
1  %i0 = arith.constant 23 : i32
2  %i1 = arith.constant 5 : i32
3  %i2 = arith.addi %i0, %i1 : i32
4
5  %cond = arith.cmpi eq, %i2, %i0 : i32
6  %false_or_true = scf.if(%cond) -> i1 {
7    %false = arith.constant false : i1
8    scf.yield %false : i1
9  } else {
10   %true = arith.constant true : i1
11   scf.yield %true : i1
12 }
```

What MLIR does for us:

- Provides the framework for creating custom representations
- Makes writing composable passes and progressive lowering easy
- Provides a set of predefined dialects (namespaces for operations) and passes

Dialects used in this example:

- arith: Provides standard arithmetic operations
- scf: Provides Structure Control Flow operations

# Physical Circuit IR

```
1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 %q0_2, %q1_2, %obs_2 = qstruct.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit) -> !qcore.qubit, !qcore.qubit, !qec.observable {
3   ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
4     qref.reset<Z>(%q0_1, %q1_1)
5     qref.gate<#qcore.gate.h>(%q0_1)
6     %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
7     qec.detector<[0.0, 0.0]>(%m0)
8     qec.detector<[0.0, 1.0]>(%m1)
9
10    %obs = qec.dec_observable -> !qec.observable
11    %obs_1 = qec.observable_include(%obs) using (%m0, %m1)
12
13    qstruct.yield %q0_1, %q1_1, %obs_1 : !qcore.qubit, !qcore.qubit, !qec.observable
14  }
15
16 %log = qec.get_corrected(%obs_2) -> i1
17 qstruct.output(%log : i1)
```



# Physical Circuit IR

```
1  %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2  %q0_2, %q1_2, %obs_2 = qstruct.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit) -> !qcore.qubit, !qcore.qubit, !qec.observable {
3  ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
4      qref.reset<Z>(%q0_1, %q1_1)
5      qref.gate<#qcore.gate.h>(%q0_1)
6      %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
7      qec.detector<[0.0, 0.0]>(%m0)
8      qec.detector<[0.0, 1.0]>(%m1)
9
10     %obs = qec.dec_observable -> !qec.observable
11     %obs_1 = qec.observable_include(%obs) using (%m0, %m1)
12
13     qstruct.yield %q0_1, %q1_1, %obs_1 : !qcore.qubit, !qcore.qubit, !qec.observable
14 }
15
16 %log = qec.get_corrected(%obs_2) -> i1
17 qstruct.output(%log : i1)
```

qcore – fundamental qubit and gate definitions

# Physical Circuit IR

```
1 %q0, %q1 = qcore.alloc qubit -> !qcore.qubit, !qcore.qubit
2 %q0_2, %q1_2, %obs_2 = qstruct.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit) -> !qcore.qubit, !qcore.qubit, !qec.observable {
3   ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit).
4     qref.reset<Z>(%q0_1, %q1_1)
5     qref.gate<#qcore.gate.h>(%q0_1)
6     %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
7     qec.detector<[0.0, 0.0]>(%m0)
8     qec.detector<[0.0, 1.0]>(%m1)
9
10    %obs = qec.dec_observable -> !qec.observable
11    %obs_1 = qec.observable_include(%obs) using (%m0, %m1)
12
13    qstruct.yield %q0_1, %q1_1, %obs_1 : !qcore.qubit, !qcore.qubit, !qec.observable
14  }
15
16 %log = qec.get_corrected(%obs_2) -> i1
17 qstruct.output(%log : i1)
```

qstruct – structuring a quantum program

# Physical Circuit IR

```
1  %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2  %q0_2, %q1_2, %obs_2 = qstruct.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit) -> !qcore.qubit, !qcore.qubit, !qec.observable {
3  ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
4      qref.reset<Z>(%q0_1, %q1_1)
5      qref.gate<#qcore.gate.h>(%q0_1)
6      %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
7      qec.detector<[0.0, 0.0]>(%m0)
8      qec.detector<[0.0, 1.0]>(%m1)
9
10     %obs = qec.dec_observable -> !qec.observable
11     %obs_1 = qec.observable_include(%obs) using (%m0, %m1)
12
13     qstruct.yield %q0_1, %q1_1, %obs_1 : !qcore.qubit, !qcore.qubit, !qec.observable
14 }
15
16 %log = qec.get_corrected(%obs_2) -> i1
17 qstruct.output(%log : i1)
```

qref – physical operations on qubit references

# Physical Circuit IR

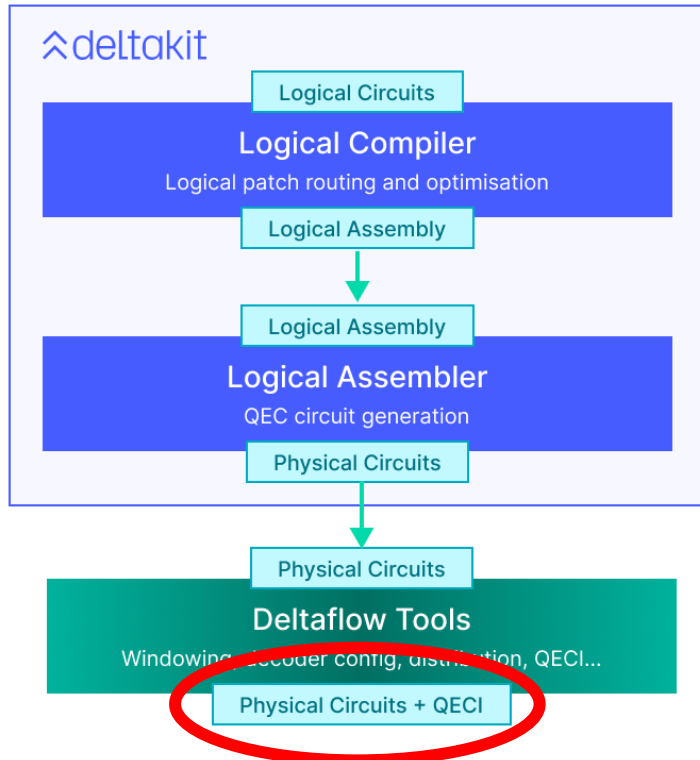
```
1  %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2  %q0_2, %q1_2, %obs_2 = qstruct.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit) -> !qcore.qubit, !qcore.qubit, !qec.observable {
3  ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
4      qref.reset<Z>(%q0_1, %q1_1)
5      qref.gate<#qcore.gate.h>(%q0_1)
6      %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
7      qec.detector<[0.0, 0.0]>(%m0)
8      qec.detector<[0.0, 1.0]>(%m1)
9
10     %obs = qec.dec_observable -> !qec.observable
11     %obs_1 = qec.observable_include(%obs) using (%m0, %m1)
12
13     qstruct.yield %q0_1, %q1_1, %obs_1 : !qcore.qubit, !qcore.qubit, !qec.observable
14 }
15
16 %log = qec.get_corrected(%obs_2) -> i1
17 qstruct.output(%log . i1)
```

qec – error correction related operations

# Physical Circuit IR with control flow

```
1 // Qubit and observables from previous circuit
2 %q0, %obs0, %obs1 = ...
3
4 // Conditional on observable 0
5 %log0 = qec.get_corrected(%obs0) -> i1
6 %q0_3, %obs1_4 = scf.if(%log0) -> !qcore.qubit, !qec.observable {
7
8     // Conditional chunk of circuit
9     %q0_2, %obs1_3 = qstruct.circuit(%q0, %obs1 : !qcore.qubit, !qec.observable) -> !qcore.qubit, !qec.observable {
10     ^bb0(%q0_1, %obs1_1 : !qcore.qubit, !qec.observable):
11         qref.gate<qcore.gate.h>(%q0_1)
12         %m = qref.measure<Z>(%q0_1) -> i1
13         %obs1_2 = qec.observable_include(%obs1_1) using (%m)
14         qstruct.yield %q0_1, %obs1_2 : !qcore.qubit, !qec.observable
15     }
16
17     // Yield modified observable 1 and qubit
18     scf.yield %q0_2, %obs1_3 : !qcore.qubit, !qec.observable
19 } else {
20     // Yield unmodified observable 1 and qubit from before the if
21     scf.yield %q0, %obs1 : !qcore.qubit, !qec.observable
22 }
23
24 // Return the final value of observable 1
25 %log1 = qec.get_corrected(%obs1_4) -> i1
26 qstruct.output(%log1 : i1)
27
```

# Physical Circuit IR with QECi

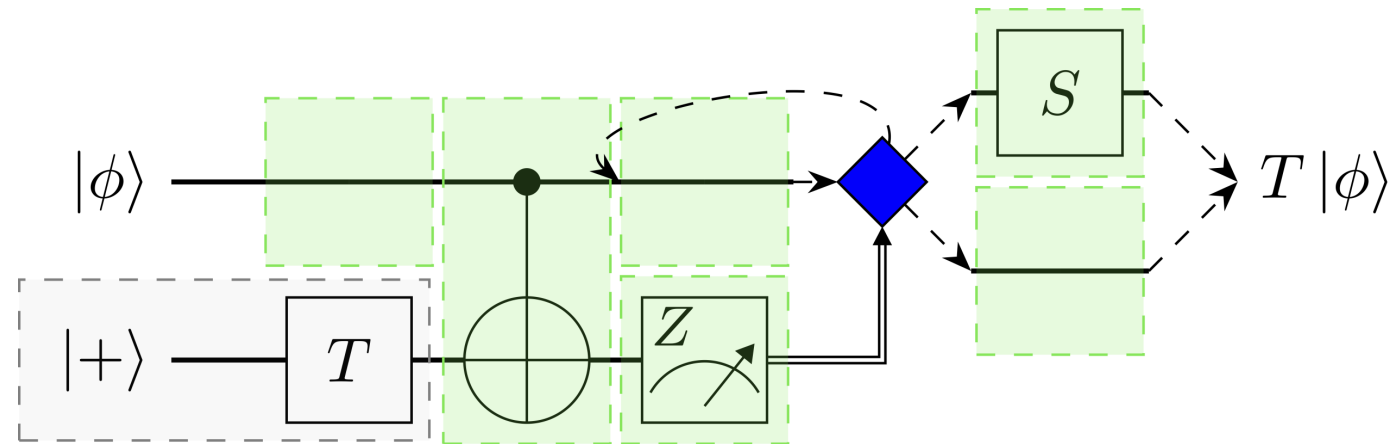


```

1  // Qubit from previous circuit
2  %q0 = ...
3
4  // Conditional on observable 0, retrieved from Deltaflow
5  %r0 = qeci.csr_read p0.0.0 -> l0.0.0 PROTOCOL OB_G_OBT_0
6  %c15 = arith.constant 15 : i64
7  %r0_1 = arith.shrui %r0, %c15 : i64
8  %log0 = arith.trunci %r0_1 : i64 to i1
9  %q0_3 = scf.if(%log0) -> !qcore.qubit {
10
11     // Conditional chunk of circuit
12     %q0_2, %obs1_3 = qstruct.circuit(%q0 : !qcore.qubit) -> !qcore.qubit {
13         ^bb0(%q0_1 : !qcore.qubit):
14             qref.gate<#qcore.gate.h>(%q0_1)
15             %m = qref.measure<Z>(%q0_1) -> i1
16
17             // Send measurement bit to Deltaflow to include in observable 1
18             qeci.send_readouts_annotation p0.0.0 -> l0.0.0 UNICAST (%m : i1)
19
20             qstruct.yield %q0_1 : !qcore.qubit
21         }
22
23         // Yield modified qubit
24         scf.yield %q0_2 : !qcore.qubit
25     } else {
26         // Yield unmodified qubit from before the if
27         scf.yield %q0 : !qcore.qubit
28     }
29
30     // Return the final value of observable 1, retrieved from Deltaflow
31     %r1 = qeci.csr_read p0.0.0 -> l0.0.0 PROTOCOL OB_G_OBT_1
32     %r1_1 = arith.shrui %r1, %c15 : i64
33     %log1 = arith.trunci %r1_1 : i64 to i1
34     qstruct.output(%log1 : i1)
  
```

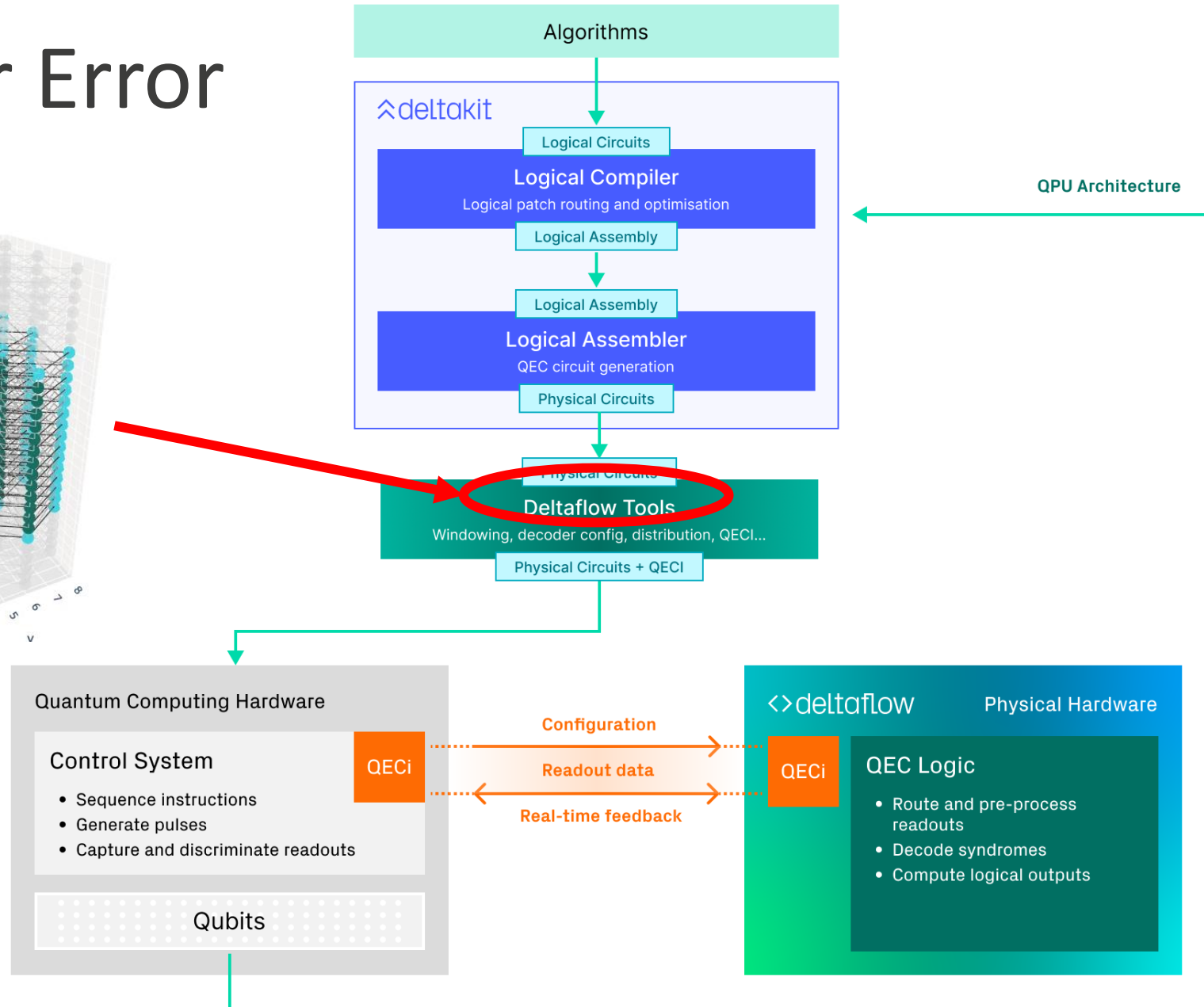
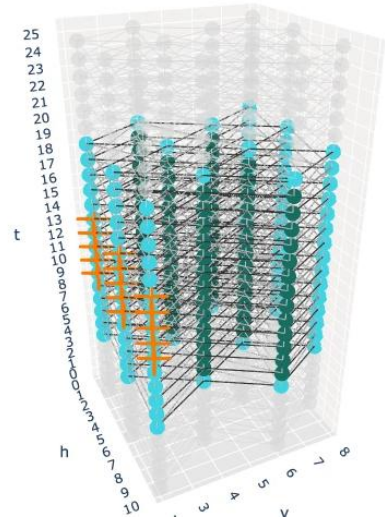
# Conclusions

- Quantum computing needs real time error correction
- Real time error correction needs to be low latency
- Quantum error correction needs to handle complex control-flow problems
- Classical compilation tools help us to solve this problem sustainably



# Dynamic DEMs

# Detector Error Model

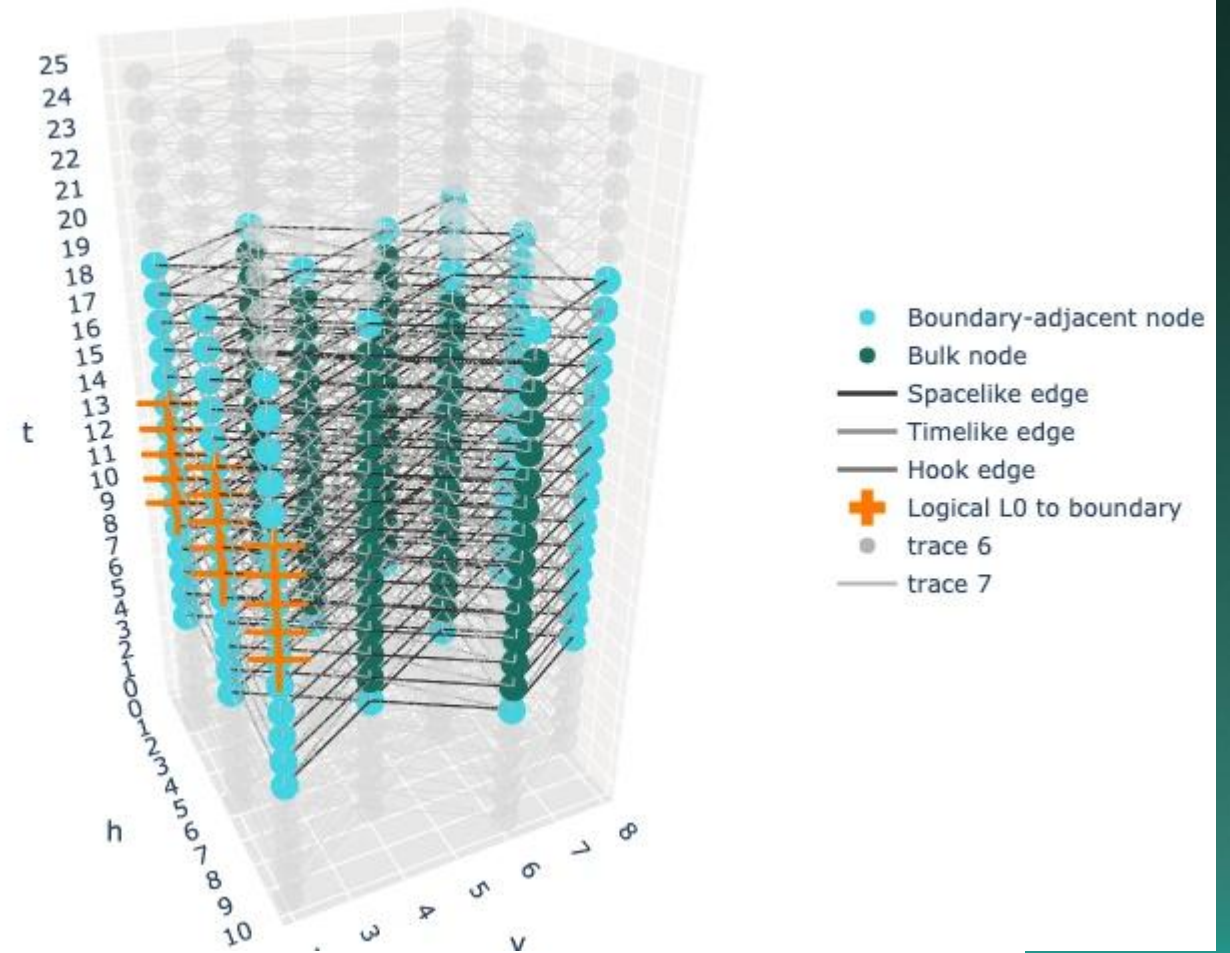


# A typical DEM representation

A (hyper)graph where:

- Nodes are detectors
- Edges are error mechanisms
- Arbitrary data can be attached to nodes/edges

Is this a complete representation of the problem?



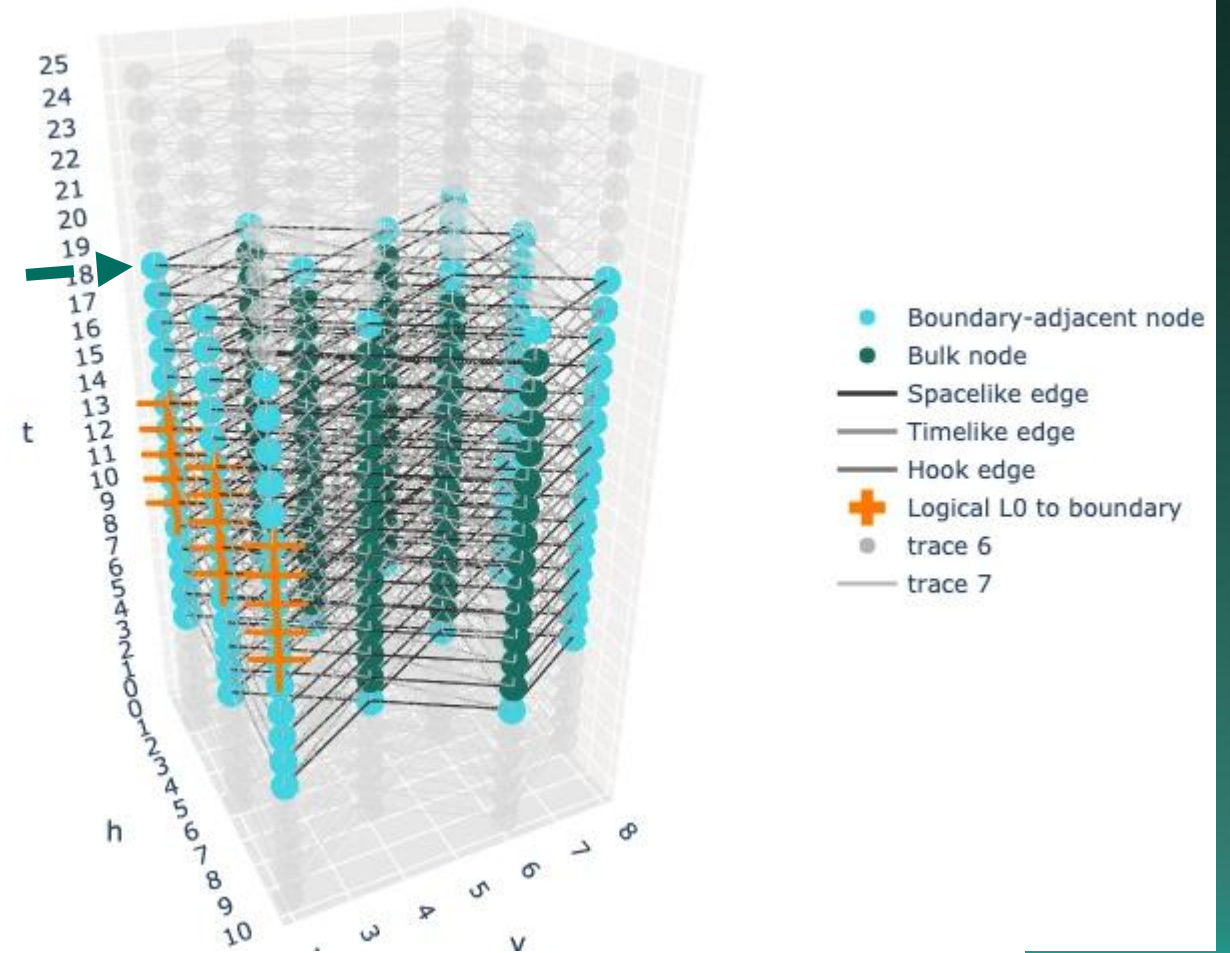
# A typical DEM representation

```
CZ 0 1 3 5 7 6 9 10
DEPOLARIZE1(0.001) 0 3
M 0 3 7 9
DETECTOR rec[-1] rec[-5]
```

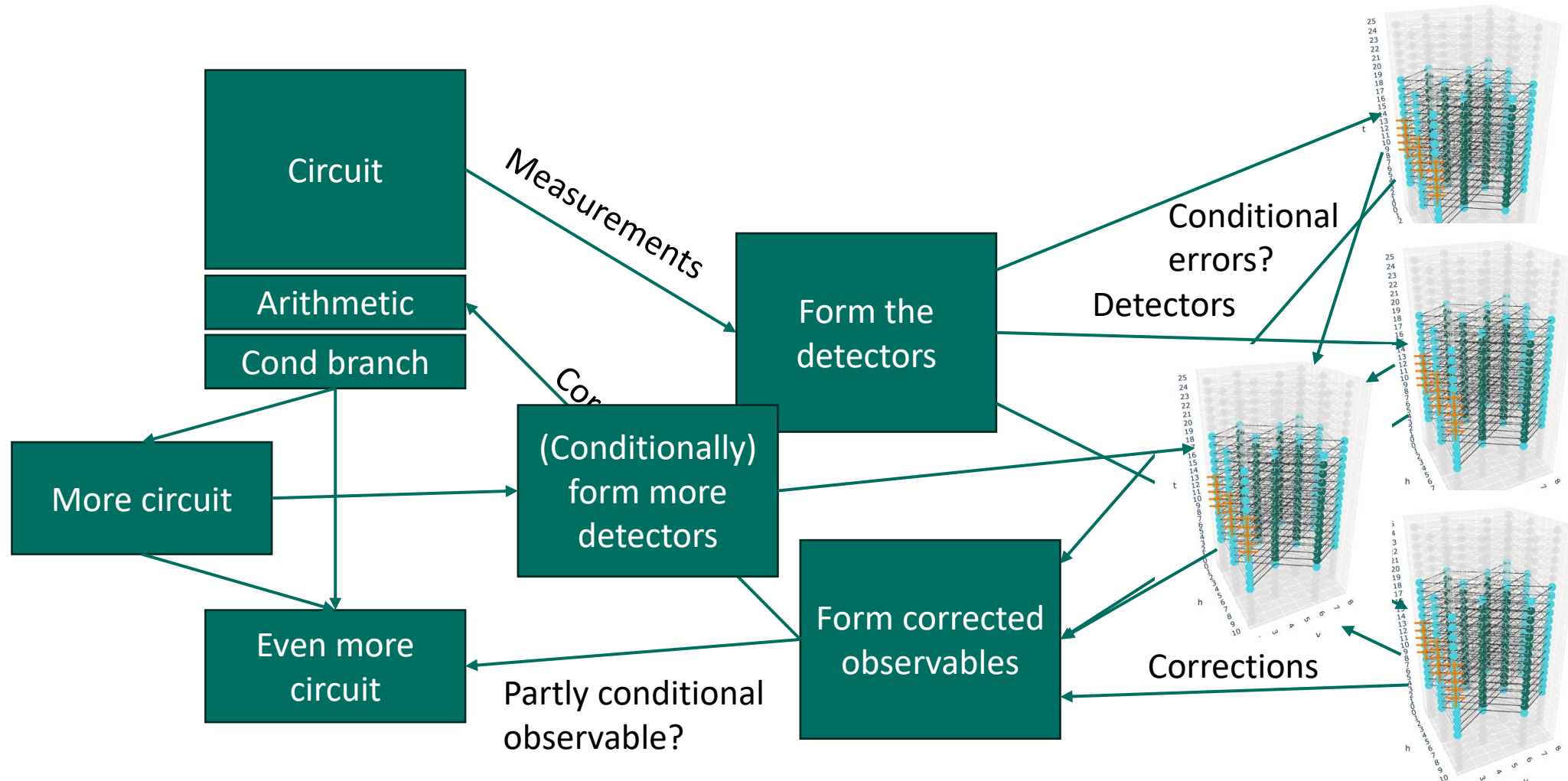
Indices of the detectors  
happen to be the same

Is this a complete representation of the problem?

Not really – an implicit relationship with the physical  
circuit is required



# A complete fault tolerant program



# Separating out the DEM

```
1  %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2  %q0_2, %q1_2, %uncorr_1, %corr_1 = dem.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit)
3  |      -> !qcore.qubit, !qcore.qubit, i1, i1 {
4  ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
5  |      qref.reset<Z>(%q0_1, %q1_1)
6  |      qref.gate<#qcore.gate.h>(%q0_1)
7  |      %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
8  |
9  |      %obs_uncorr = qec.dec_observable -> !qec.observable
10 |      %obs_uncorr_1 = qec.observable_include(%obs_uncorr) using (%m0, %m1)
11 |      %uncorr = qec.get_uncorrected(%obs_uncorr_1) -> i1
12 |
13 |      dem.yield to_dem(%m0, %m1 : i1, i1) %q0_1, %q1_1, %uncorr : !qcore.qubit, !qcore.qubit, i1
14 | } {
15 ^bb0(%m0_1, %m1_1 : i1, i1):
16 | %obs_corr_2 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable {
17 | ^bb0(%m0_2, %m1_2 : i1, i1):
18 | | %obs_corr = qec.dec_observable -> !qec.observable
19 | |
20 | | %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
21 | | %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
22 | | %obs_corr_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
23 | |
24 | | dem.yield %obs_corr_1 : !qec.observable
25 | | }
26 | %corr = qec.get_correction(%obs_corr_2) -> i1
27 | dem.yield %corr : i1
28 | }
29
30 %log = arith.xor %uncorr_1, %corr_1 : i1
31 qstruct.output(%log : i1)
```

# Separating out the DEM

```
1  %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2  %q0_2, %q1_2, %uncorr_1, %corr_1 = dem.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit)
3  -> !qcore.qubit, !qcore.qubit, i1, i1 {
4  ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
5      qref.reset<Z>(%q0_1, %q1_1)
6      qref.gate<#qcore.gate.h>(%q0_1)
7      %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
8
9      %obs_uncorr = qec.dec_observable -> !qec.observable
10     %obs_uncorr_1 = qec.observable_include(%obs_uncorr) using (%m0, %m1)
11     %uncorr = qec.get_uncorrected(%obs_uncorr_1) -> i1
12
13     dem.yield to dem(%m0, %m1 : i1, i1) %q0_1, %q1_1, %uncorr : !qcore.qubit, !qcore.qubit, i1
14 } {
15 ^bb0(%m0_1, %m1_1 : i1, i1):
16     %obs_corr_2 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable {
17     ^bb0(%m0_2, %m1_2 : i1, i1):
18         %obs_corr = qec.dec_observable -> !qec.observable
19
20         %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
21         %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
22         %obs_corr_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
23
24         dem.yield %obs_corr_1 : !qec.observable
25     }
26     %corr = qec.get_correction(%obs_corr_2) -> i1
27     dem.yield %corr : i1
28 }
29
30 %log = arith.xor %uncorr_1, %corr_1 : i1
31 qstruct.output(%log : i1)
```

Physical circuit region

DEM region

# Separating out the DEM

```
1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 %q0_2, %q1_2, %uncorr_1, %corr_1 = dem.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit)
3     -> !qcore.qubit, !qcore.qubit, i1, i1 {
4 ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
5     qref.reset<Z>(%q0_1, %q1_1)
6     qref_gate<#qcore.gate.h>(%q0_1)
7     %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
8
9     %obs_uncorr = qec.dec_observable -> !qec.observable
10    %obs_uncorr_1 = qec.observable_include(%obs_uncorr) using (%m0, %m1)
11    %uncorr = qec.get_uncorrected(%obs_uncorr_1) -> i1
12
13    dem.yield to_dem(%m0, %m1 : i1, i1) %q0_1, %q1_1, %uncorr : !qcore.qubit, !qcore.qubit, i1
14 } {
15 ^bb1(%m0_1, %m1_1 : i1, i1):
16    %obs_corr_2 = dem.det(%m0_1, %m1_1 : i1, i1) -> !qec.observable {
17    ^bb2(%m0_2, %m1_2 : i1, i1):
18        %obs_corr = qec.dec_observable -> !qec.observable
19
20        %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
21        %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
22        %obs_corr_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
23
24        dem.yield %obs_corr_1 : !qec.observable
25    }
26    %corr = qec.get_correction(%obs_corr_2) -> i1
27    dem.yield %corr : i1
28 }
29
30 %log = arith.xor %uncorr_1, %corr_1 : i1
31 qstruct.output(%log : i1)
```

Measurements passed to DEM

Measurements passed through  
into detectors

# Separating out the DEM

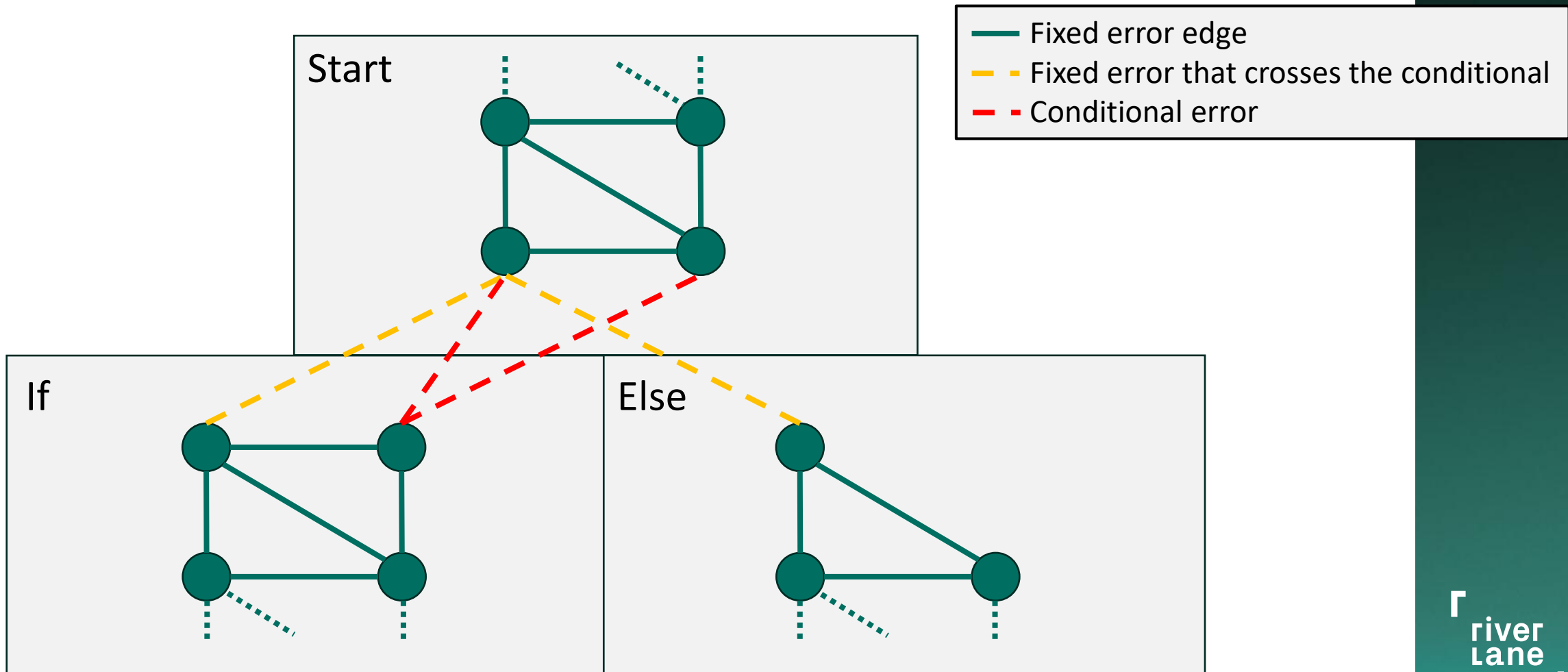
```
1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 %q0_2, %q1_2, %uncorr_1, %corr_1 = dem.circuit(%q0, %q1 : !qcore.qubit, !qcore.qubit)
3     -> !qcore.qubit, !qcore.qubit, i1, i1 {
4 ^bb0(%q0_1, %q1_1 : !qcore.qubit, !qcore.qubit):
5     qref.reset<Z>(%q0_1, %q1_1)
6     qref.gate<#qcore.gate.h>(%q0_1)
7     %m0, %m1 = qref.measure<Z>(%q0_1, %q1_1) -> i1, i1
8
9     %obs_uncorr = qec.dec_observable -> !qec.observable
10    %obs_uncorr_1 = qec.observable_include(%obs_uncorr) using (%m0, %m1)
11    %uncorr = qec.get_uncorrected(%obs_uncorr_1) -> i1
12
13    dem.yield to_dem(%m0, %m1 : i1, i1) %q0_1, %q1_1, %uncorr : !qcore.qubit, !qcore.qubit, i1
14 } {
15 ^bb0(%m0_1, %m1_1 : i1, i1):
16     %obs_corr_2 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable {
17     ^bb0(%m0_2, %m1_2 : i1, i1):
18         %obs_corr = qec.dec_observable -> !qec.observable
19
20         %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
21         %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
22         %obs_corr_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
23
24         dem.yield %obs_corr_1 : !qec.observable
25     }
26     %corr = qec.get_correction(%obs_corr_2) -> i1
27     dem.yield %corr : i1
28 }
29
30 %log = arith.xor %uncorr_1, %corr_1 : i1
31 astruct.output(%log : i1)
```

Form the uncorrected observable

Form the logical correction

XOR them together to form the corrected observable

# Conditional errors



```

1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 ... %obs_3, %d0_2 = dem.circuit(...) -> ... !qec.observable, !dem.detector_ref {
3     ...
4 } {
5 ^bb0(%m0_1, %m1_1 : i1, i1):
6     %obs_2, %d0_1 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable, !dem.detector_ref {
7     ^bb0(%m0_2, %m1_2 : i1, i1):
8         %obs_corr = qec.dec_observable -> !qec.observable
9
10         %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
11         %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
12         %obs_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
13
14         dem.yield %obs_1, %d0 : !qec.observable, !dem.detector_ref
15     }
16     dem.yield %obs_2, %d0_1 : !qec.observable, !dem.detector_ref
17 }
18
19 %obs_9 = scf.if(...) -> !qec.observable {
20     %obs_8 = dem.circuit(...) to_dem(%obs_3, %d0_2 : !qec.observable, !dem.detector_ref) -> !qec.observable {
21         ...
22     } {
23     ^bb0(%m0_2, %m1_2, %m2_2, %m3_2, %obs_4, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
24         %obs_7 = dem.dem(%m0_2, %m1_2, %m2_2, %m3_2, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref) -> !qec.observable {
25         ^bb0(%m0_3, %m1_3, %m2_3, %m3_3, %obs_5, %d0_4 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
26
27             %d2 = dem.detector<[0.0, 0.0]>(%m0_3, %m2_3)
28             %obs_6 = dem.error<0.001>(%d0_4, %d2, %obs_5)
29
30             dem.yield %obs_6 : !qec.observable
31         }
32         dem.yield %obs_7 : !qec.observable
33     }
34     scf.yield %obs_8 : !qec.observable
35 } else {
36     scf.yield %obs_3 : !qec.observable
37 }

```

```

1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 ... %obs_3, %d0_2 = dem.circuit(...) -> ... !qec.observable, !dem.detector_ref {
3   ...
4 } {
5 ^bb0(%m0_1, %m1_1 : i1, i1):
6   %obs_2, %d0_1 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable, !dem.detector_ref {
7     ^bb0(%m0_2, %m1_2 : i1, i1):
8       %obs_corr = qec.dec_observable -> !qec.observable
9
10      %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
11      %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
12      %obs_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
13
14      dem.yield %obs_1, %d0 : !qec.observable, !dem.detector_ref
15    }
16    dem.yield %obs_2, %d0_1 : !qec.observable, !dem.detector_ref
17  }
18
19 %obs_9 = scf.if(...) -> !qec.observable {
20   %obs_8 = dem.circuit(...) to_dem(%obs_3, %d0_2 : !qec.observable, !dem.detector_ref) -> !qec.observable {
21     ...
22   } {
23     ^bb0(%m0_2, %m1_2, %m2_2, %m3_2, %obs_4, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
24       %obs_7 = dem.dem(%m0_2, %m1_2, %m2_2, %m3_2, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref) -> !qec.observable {
25         ^bb0(%m0_3, %m1_3, %m2_3, %m3_3, %obs_5, %d0_4 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
26
27           %d2 = dem.detector<[0.0, 0.0]>(%m0_3, %m2_3)
28           %obs_6 = dem.error<0.001>(%d0_4, %d2, %obs_5)
29
30           dem.yield %obs_6 : !qec.observable
31         }
32         dem.yield %obs_7 : !qec.observable
33       }
34       scf.yield %obs_8 : !qec.observable
35     } else {
36       scf.yield %obs_3 : !qec.observable
37     }

```

Starting DEM

Classical conditional yielding %obs\_9

Conditional DEM

Value of %obs\_9 is dependent on the path chosen

```

1 %q0, %q1 = qcore.alloc_qubit -> !qcore.qubit, !qcore.qubit
2 ... %obs_3, %d0_2 = dem.circuit(...) -> ... !qec.observable, !dem.detector_ref {
3   ...
4 } {
5   ^bb0(%m0_1, %m1_1 : i1, i1):
6     %obs_2, %d0_1 = dem.dem(%m0_1, %m1_1 : i1, i1) -> !qec.observable, !dem.detector_ref {
7       ^bb0(%m0_2, %m1_2 : i1, i1):
8         %obs_corr = qec.dec_observable -> !qec.observable
9
10        %d0 = dem.detector<[0.0, 0.0]>(%m0_2)
11        %d1 = dem.detector<[0.0, 1.0]>(%m1_2)
12        %obs_1 = dem.error<0.001>(%d0, %d1, %obs_corr)
13
14        dem.yield %obs_1, %d0 : !qec.observable, !dem.detector_ref
15      }
16      dem.yield %obs_2, %d0_1 : !qec.observable, !dem.detector_ref
17    }
18
19    %obs_9 = scf.if(...) -> !qec.observable {
20      %obs_8 = dem.circuit(...) to_dem(%obs_3, %d0_2 : !qec.observable, !dem.detector_ref) -> !qec.observable {
21        ...
22      } {
23        ^bb0(%m0_2, %m1_2, %m2_2, %m3_2, %obs_4, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
24          %obs_7 = dem.dem(%m0_2, %m1_2, %m2_2, %m3_2, %d0_3 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref) -> !qec.observable {
25            ^bb0(%m0_3, %m1_3, %m2_3, %m3_3, %obs_5, %d0_4 : i1, i1, i1, i1, !qec.observable, !dem.detector_ref):
26
27              %d2 = dem.detector<[0.0, 0.0]>(%m0_3, %m2_3)
28              %obs_6 = dem.error<0.001>(%d0_4, %d2, %obs_5)
29
30              dem.yield %obs_6 : !qec.observable
31            }
32            dem.yield %obs_7 : !qec.observable
33          }
34          scf.yield %obs_8 : !qec.observable
35        } else {
36          scf.yield %obs_3 : !qec.observable
37        }

```

Detector reference passed out of starting DEM

Detector reference passed to conditional DEM

A new error is formed between the passed in detector reference and a detector declared inside the conditional DEM

# Conclusions

- Dynamic DEMs may be represented alongside their corresponding physical circuits using MLIR
- This opens the door to compiling fault tolerant programs for both simulation and real hardware
- It also gives us a good baseline representation to explore more complex compilation techniques with

## Open research:

- Efficient extraction of dynamic DEMs from dynamic physical circuits
- General purpose windowing schemes that handle control flow well
- Can you make a decoder that natively understands a dynamic DEM?

Thank you!  
Questions?

river  
lane

